TRƯỜNG ĐẠI HỌC CẦN THƠ
TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG

BÁO CÁO ĐỀ TÀI

MÔN KHO DỮ LIỆU VÀ KHAI PHÁ DỮ LIỆU (CTH603)

Xây dựng mô hình dự đoán nguy cơ đột quỵ dựa trên các yếu tố lâm sàng và nhân khẩu học, so sánh hiệu quả giữa Decision Tree, k-NN và SVM

Giảng viên hướng dẫn

PGS.TS. Nguyễn Thái Nghe

Sinh viên thực hiện

Lê Văn Triệu MSSV: M2526034

La Quỳnh Như MSSV: M2526026

Cần Thơ – 2026

Mục lục

Tóm tắt **3**

Abstract **4**

1 Giới thiệu **5**
1.1 Đặt vấn đề 5
1.2 Mục tiêu đề tài 5
1.3 Phạm vi đề tài 6

2 Cơ sở lý thuyết **7**
2.1 Decision Tree (Cây quyết định) 7
2.2 k-Nearest Neighbors (k-NN) 7
2.3 Support Vector Machine (SVM) 7
2.4 Các độ đo đánh giá mô hình 8
2.5 Cross-Validation (Kiểm định chéo) 8

3 Dữ liệu và phương pháp thực hiện **9**
3.1 Bộ dữ liệu 9
3.1.1 Thống kê mô tả 9
3.1.2 Tiền xử lý dữ liệu 10
3.2 Phương pháp chia dữ liệu 10
3.3 Phương pháp dò tham số và xử lý mất cân bằng lớp 10
3.4 Phương pháp đánh giá cuối cùng 11

4 Kết quả thực nghiệm và xây dựng hệ thống demo **12**
4.1 Kết quả dò tham số từng mô hình 12
4.1.1 Decision Tree 12
4.1.2 k-Nearest Neighbors 12
4.1.3 Support Vector Machine 12
4.2 So sánh tổng hợp và kết luận chính 13
4.3 Kiểm chứng bằng 5-fold Cross-Validation 13
4.4 Xây dựng hệ thống demo 14

4.4.1 Kiến trúc hệ thống 14
4.4.2 Các chức năng của hệ thống demo 14

5 Kết luận và hướng phát triển **20**
5.1 Kết luận 20
5.2 Hạn chế 20
5.3 Hướng phát triển 21

Tóm tắt

Đột quỵ là một trong những nguyên nhân gây tử vong và tàn tật hàng đầu trên thế giới, trong khi việc phát hiện sớm nguy cơ có thể giúp can thiệp y tế kịp thời. Đồ án này xây dựng và so sánh ba mô hình học máy – Decision Tree, k-Nearest Neighbors (k-NN) và Support Vector Machine (SVM) – để dự đoán nguy cơ đột quỵ dựa trên các yếu tố lâm sàng và nhân khẩu học, sử dụng bộ dữ liệu Stroke Prediction Dataset (5.110 bệnh nhân, 12 thuộc tính) từ Kaggle. Do dữ liệu mất cân bằng lớp nặng (chỉ 4,87% bệnh nhân có đột quỵ), đồ án tập trung xử lý vấn đề này bằng `class_weight='balanced'` và điều chỉnh ngưỡng quyết định, đồng thời đánh giá bằng nhiều độ đo (Accuracy, Precision, Recall, F1, AUC, Confusion Matrix) thay vì chỉ dựa vào Accuracy. Sau khi dò tham số tối ưu cho từng mô hình và kiểm chứng bằng 5-fold Cross-Validation, kết quả cho thấy **SVM (kernel tuyến tính, C=0,01) đạt hiệu quả tốt nhất** với AUC = 0,8404 (một lần chia train/test) và AUC = 0,8379 $\pm$ 0,0249 (5-fold CV), vượt trội so với Decision Tree (AUC = 0,8305) và k-NN (AUC = 0,7831). Đồ án còn xây dựng một hệ thống demo đầy đủ (Backend Python/FastAPI, Frontend Next.js) cho phép người dùng nhập thông tin bệnh nhân để dự đoán, xem và tự điều chỉnh tham số mô hình, cũng như chạy lại trực tiếp quá trình dò tham số để kiểm chứng kết quả.

Từ khóa: dự đoán đột quỵ, Decision Tree, k-Nearest Neighbors, Support Vector Machine, mất cân bằng lớp, cross-validation.

Abstract

Stroke is one of the leading causes of death and disability worldwide, and early risk detection can enable timely medical intervention. This project builds and compares three machine learning models – Decision Tree, k-Nearest Neighbors (k-NN), and Support Vector Machine (SVM) – to predict stroke risk based on clinical and demographic factors, using the Stroke Prediction Dataset (5,110 patients, 12 attributes) from Kaggle. Given the severe class imbalance in the data (only 4.87% of patients had a stroke), the project focuses on handling this issue via `class_weight='balanced'` and decision-threshold tuning, and evaluates models using multiple metrics (Accuracy, Precision, Recall, F1, AUC, Confusion Matrix) rather than Accuracy alone. After hyperparameter tuning and verification via 5-fold Cross-Validation, results show that **SVM (linear kernel, C=0.01) performs best**, achieving AUC = 0.8404 (single train/test split) and AUC = 0.8379 $\pm$ 0.0249 (5-fold CV), outperforming Decision Tree (AUC = 0.8305) and k-NN (AUC = 0.7831). The project also delivers a full demo system (Python/FastAPI backend, Next.js frontend) that lets users enter patient information for prediction, view and adjust model parameters, and re-run the parameter search live to verify the results.

Keywords: stroke prediction, Decision Tree, k-Nearest Neighbors, Support Vector Machine, class imbalance, cross-validation.

Chương 1

Giới thiệu

1.1 Đặt vấn đề

Theo Tổ chức Y tế Thế giới (WHO), đột quỵ là nguyên nhân gây tử vong đứng thứ hai trên toàn cầu và là nguyên nhân hàng đầu gây tàn tật lâu dài ở người trưởng thành. Việc phát hiện sớm các yếu tố nguy cơ – như tuổi tác, tăng huyết áp, bệnh tim, mức glucose trong máu, chỉ số khối cơ thể (BMI), thói quen hút thuốc – có ý nghĩa quan trọng trong việc hỗ trợ bác sĩ sàng lọc và tư vấn can thiệp kịp thời cho bệnh nhân có nguy cơ cao.

Học máy (machine learning) là công cụ phù hợp để khai thác mối quan hệ giữa nhiều yếu tố lâm sàng/nhân khẩu học và nguy cơ đột quỵ, vốn khó mô hình hóa bằng các quy tắc thủ công đơn giản. Tuy nhiên, dữ liệu y tế thực tế thường gặp một thách thức lớn: **mất cân bằng lớp** – số ca bệnh thực sự (dương tính) luôn chiếm tỉ lệ rất nhỏ so với số người khỏe mạnh. Nếu không xử lý đúng cách, mô hình có thể đạt độ chính xác (Accuracy) rất cao chỉ bằng cách luôn dự đoán ”không có nguy cơ”, trong khi trên thực tế lại không phát hiện được bất kỳ ca bệnh nào – một vấn đề được gọi là ”nghịch lý Accuracy” (Accuracy Paradox).

1.2 Mục tiêu đề tài

Đồ án đặt ra các mục tiêu sau:

1. Xây dựng và huấn luyện ba mô hình phân lớp đã học trong môn Kho dữ liệu và khai phá dữ liệu – Decision Tree, k-Nearest Neighbors (k-NN) và Support Vector Machine (SVM) – trên bộ dữ liệu dự đoán đột quỵ.

2. Dò tham số tối ưu cho từng mô hình (độ sâu cây, số K, ngưỡng quyết định, kernel và hệ số C) bằng thực nghiệm có bằng chứng số liệu cụ thể, không chọn tham số tùy tiện.

3. Xử lý đúng đắn vấn đề mất cân bằng lớp và đánh giá mô hình bằng đầy đủ các độ đo phù hợp (Precision, Recall, F1, AUC/ROC, Confusion Matrix), không chỉ dựa vào Accuracy.

4. So sánh khách quan 3 mô hình trên cùng một cách chia dữ liệu, rút ra kết luận mô hình nào phù hợp nhất cho bài toán này, và kiểm chứng lại bằng 5-fold Cross-Validation.

5. Xây dựng một hệ thống demo hoàn chỉnh (giao diện web) cho phép nhập thông tin bệnh nhân và nhận kết quả dự đoán từ cả 3 mô hình, đồng thời minh chứng trực quan quá trình dò tham số đã thực hiện.

1.3 Phạm vi đề tài

Đồ án sử dụng bộ dữ liệu công khai Stroke Prediction Dataset (Kaggle), không sử dụng dữ liệu bệnh viện thực tế do vấn đề bảo mật và quyền riêng tư. Ba mô hình được giới hạn trong phạm vi các thuật toán phân lớp đã học ở môn Kho dữ liệu và khai phá dữ liệu (Decision Tree, k-NN, SVM), không mở rộng sang các mô hình học sâu (deep learning). Hệ thống demo được xây dựng ở quy mô vừa đủ để minh họa và kiểm chứng kết quả nghiên cứu, không hướng tới triển khai thực tế trong môi trường bệnh viện.

Chương 2

Cơ sở lý thuyết

2.1 Decision Tree (Cây quyết định)

Decision Tree là thuật toán phân lớp xây dựng một cấu trúc cây, trong đó mỗi nút trong biểu diễn một phép kiểm tra trên một thuộc tính (ví dụ: ”tuổi $>$ 50?”), mỗi nhánh là kết quả của phép kiểm tra, và mỗi nút lá là một nhãn dự đoán. Cây được xây dựng bằng cách chọn thuộc tính phân chia tốt nhất tại mỗi bước dựa trên tiêu chí làm giảm độ hỗn tạp (impurity) của tập dữ liệu con, ví dụ chỉ số Gini hoặc Entropy. Ưu điểm của Decision Tree là dễ diễn giải (có thể đọc trực tiếp các luật quyết định), nhưng nhược điểm lớn là dễ **overfitting** (học quá khớp) nếu cây mọc quá sâu, khiến mô hình học theo nhiễu của từng điểm dữ liệu cụ thể thay vì quy luật tổng quát. Tham số quan trọng cần kiểm soát là `max_depth` (độ sâu tối đa của cây).

2.2 k-Nearest Neighbors (k-NN)

k-NN là thuật toán phân lớp dựa trên khoảng cách: để dự đoán nhãn của một điểm dữ liệu mới, thuật toán tìm K điểm dữ liệu ”gần” nhất trong tập huấn luyện (theo một độ đo khoảng cách, thường là khoảng cách Euclid), rồi lấy nhãn chiếm đa số trong K điểm đó làm kết quả dự đoán. k-NN không thực sự ”học” một hàm tham số nào trong quá trình huấn luyện (gọi là thuật toán ”lazy learning” – chỉ ghi nhớ dữ liệu), mà toàn bộ việc tính toán diễn ra lúc dự đoán. Vì dựa trên khoảng cách, k-NN **nhạy cảm với thang đo của các thuộc tính** – thuộc tính có khoảng giá trị lớn hơn sẽ chi phối khoảng cách tính toán nếu dữ liệu chưa được chuẩn hóa (ví dụ `avg_glucose_level` dao động 55–272 so với `hypertension` chỉ 0 hoặc 1) – nên dữ liệu đầu vào cần được chuẩn hóa (StandardScaler) trước khi áp dụng.

2.3 Support Vector Machine (SVM)

SVM tìm một siêu phẳng (hyperplane) phân tách hai nhóm dữ liệu sao cho khoảng cách (margin) từ siêu phẳng đến các điểm dữ liệu gần nhất của mỗi nhóm (gọi là support vector) là lớn nhất.

Khi dữ liệu không thể phân tách tuyến tính, SVM có thể dùng các hàm nhân (kernel) như `rbf` để ánh xạ dữ liệu sang không gian nhiều chiều hơn, nơi việc phân tách tuyến tính trở nên khả thi. Tham số `C` điều khiển mức độ "chấp nhận sai" trong lúc huấn luyện: `C` lớn ép mô hình phân loại đúng mọi điểm huấn luyện (dễ overfitting), `C` nhỏ cho phép một số điểm bị phân loại sai để đổi lấy một ranh giới tổng quát hơn. Giống Decision Tree, SVM hỗ trợ trực tiếp tham số `class_weight` để xử lý mất cân bằng lớp ngay trong quá trình huấn luyện.

2.4 Các độ đo đánh giá mô hình

- **Accuracy**: tỉ lệ dự đoán đúng trên tổng số mẫu. Không đáng tin cậy khi dữ liệu mất cân bằng lớp nặng.
- **Precision**: trong số các ca được dự đoán "có đột quỵ", bao nhiêu phần trăm thực sự đúng.
- **Recall**: trong số các ca thực sự "có đột quỵ", mô hình phát hiện đúng được bao nhiêu phần trăm – độ đo quan trọng nhất trong bài toán y tế vì bỏ sót một ca bệnh thật (False Negative) nguy hiểm hơn nhiều so với báo động giả.
- **F1-score**: trung bình điều hòa (harmonic mean) giữa Precision và Recall, cân bằng cả hai.
- **AUC (Area Under the ROC Curve)**: đo khả năng *xếp hạng* của mô hình (phân biệt ai có nguy cơ cao hơn ai) trên toàn bộ các ngưỡng quyết định có thể, không phụ thuộc vào một ngưỡng cụ thể. AUC = 0,5 tương đương đoán ngẫu nhiên, AUC = 1,0 là phân biệt hoàn hảo.
- **Confusion Matrix (ma trận nhầm lẫn)**: bảng 2×2 thể hiện số lượng True Positive (TP), True Negative (TN), False Positive (FP), False Negative (FN) – là cơ sở để tính mọi độ đo ở trên.

2.5 Cross-Validation (Kiểm định chéo)

Kiểm định chéo k-fold là kỹ thuật chia dữ liệu thành k phần bằng nhau, lần lượt dùng $k-1$ phần để huấn luyện và 1 phần còn lại để kiểm tra, lặp lại k lần sao cho mỗi phần đều được dùng để kiểm tra đúng một lần. Kết quả cuối cùng là trung bình và độ lệch chuẩn của các độ đo qua k lần lặp. Mục đích chính là kiểm chứng rằng kết quả đánh giá mô hình không phải do may rủi của một cách chia dữ liệu cụ thể, mà ổn định qua nhiều cách chia khác nhau. Với dữ liệu mất cân bằng lớp, cần dùng **Stratified k-fold** thay vì k-fold thông thường, để đảm bảo mỗi phần đều giữ đúng tỉ lệ lớp dương tính, tránh trường hợp một phần không có ca bệnh nào.

Chương 3

Dữ liệu và phương pháp thực hiện

3.1 Bộ dữ liệu

Đồ án sử dụng bộ dữ liệu **Stroke Prediction Dataset**, tác giả fedesoriano, công bố công khai trên Kaggle[1]. Tác giả ghi nguồn gốc dữ liệu là "Confidential Source" (không công bố bệnh viện/tổ chức cụ thể cung cấp) – đây là bộ dữ liệu công khai phổ biến trong học thuật, không phải dữ liệu chính thống từ một cơ quan y tế cụ thể.

Bộ dữ liệu gồm **5.110 bệnh nhân** và **12 thuộc tính**, trình bày ở Bảng 3.1.

Bảng 3.1: Cấu trúc bộ dữ liệu

Thuộc tính	**Ý nghĩa**
id	Mã bệnh nhân (loại bỏ khi huấn luyện)
gender	Giới tính
age	Tuổi
hypertension	Tăng huyết áp (0/1)
heart_disease	Bệnh tim (0/1)
ever_married	Tình trạng hôn nhân
work_type	Loại công việc
Residence_type	Nơi ở (thành thị/nông thôn)
avg_glucose_level	Mức glucose trung bình trong máu
bmi	Chỉ số khối cơ thể (có giá trị thiếu)
smoking_status	Tình trạng hút thuốc
stroke	**Nhãn mục tiêu** – 1 = có đột quỵ, 0 = không

3.1.1 Thống kê mô tả

Khảo sát thực tế trên dữ liệu cho thấy:

- Cột `bmi` thiếu 201/5.110 giá trị (3,9%).

[1] `https://www.kaggle.com/datasets/fedesoriano/stroke-prediction-dataset`

- Nhãn `stroke` mất cân bằng lớp **rất nặng**: 4.861 người không đột quỵ (95,13%) so với chỉ 249 người có đột quỵ (4,87%).
- 5 thuộc tính dạng chữ (`gender`, `ever_married`, `work_type`, `Residence_type`, `smoking_status`) cần được mã hóa thành số trước khi đưa vào mô hình.

Đây chính là lý do đồ án phải báo cáo đầy đủ Precision/Recall/F1/AUC thay vì chỉ Accuracy: một mô hình luôn dự đoán "không đột quỵ" vẫn đạt Accuracy $\approx 95\%$ nhưng hoàn toàn vô dụng về mặt y khoa.

3.1.2 Tiền xử lý dữ liệu

Các bước tiền xử lý được thực hiện gồm:

1. **Điền giá trị thiếu**: 201 ô trống ở cột `bmi` được điền bằng giá trị trung vị (median = 28,1) thay vì trung bình, vì trung vị ít bị ảnh hưởng bởi các giá trị BMI bất thường, phù hợp hơn với dữ liệu y tế vốn dễ có ngoại lệ.
2. **Loại bỏ cột** `id`: chỉ là mã định danh, không mang thông tin y khoa, có thể khiến mô hình học sai lệch theo thứ tự.
3. **Mã hóa biến phân loại** (Label Encoding) cho 5 cột dạng chữ – chuyển mỗi giá trị chữ thành một số nguyên tương ứng (ví dụ `gender`: Male=0, Female=1, Other=2). Các con số này chỉ mang tính ký hiệu phân loại, không thể hiện thứ tự hơn kém.

Sau tiền xử lý, dữ liệu còn 5.110 dòng, 11 cột, không còn ô trống hay cột dạng chữ, sẵn sàng đưa vào huấn luyện.

3.2 Phương pháp chia dữ liệu

Toàn bộ 3 mô hình được huấn luyện và đánh giá trên **cùng một cách chia dữ liệu** để đảm bảo so sánh công bằng: 80% huấn luyện (4.088 dòng) / 20% kiểm tra (1.022 dòng), dùng `train_test_split(..., test_size=0.2, random_state=42, stratify=y)`. Tham số `stratify=y` đảm bảo tỉ lệ nhãn `stroke=1` ($\approx$ 4,87–4,89%) được giữ đều ở cả tập huấn luyện và tập kiểm tra, tránh trường hợp tập kiểm tra ngẫu nhiên có quá ít hoặc quá nhiều ca đột quỵ.

3.3 Phương pháp dò tham số và xử lý mất cân bằng lớp

Mỗi mô hình được dò nhiều giá trị tham số khác nhau trên cùng tập dữ liệu, chọn cấu hình tốt nhất theo tiêu chí AUC cao nhất (phù hợp với dữ liệu mất cân bằng, không phụ thuộc một ngưỡng cụ thể):

- **Decision Tree**: dò `max_depth` từ 1 đến 20 (và không giới hạn), kết hợp `class_weight='balanced'`.
- **k-NN**: dò K từ 1 đến 30, với 2 cách tính trọng số hàng xóm (`uniform`, `distance`), trên dữ liệu đã chuẩn hóa; sau đó dò thêm ngưỡng quyết định (decision threshold) vì k-NN không hỗ trợ `class_weight`.
- **SVM**: dò `kernel` (`linear`, `rbf`) $\times$ `C` (0,01 đến 10), kết hợp `class_weight='balanced'`, trên dữ liệu đã chuẩn hóa.

Decision Tree và SVM xử lý mất cân bằng lớp trực tiếp qua `class_weight='balanced'` (tác động vào hàm mục tiêu lúc huấn luyện). k-NN không có cơ chế tương tự (vì đây là thuật toán "ghi nhớ" dữ liệu, không học tham số), nên được bù lại bằng cách **hạ ngưỡng quyết định** sau khi huấn luyện, nhằm đạt mức Recall tương đương hai mô hình còn lại để so sánh công bằng.

3.4 Phương pháp đánh giá cuối cùng

Sau khi chốt cấu hình tốt nhất cho từng mô hình, đồ án thực hiện hai lớp kiểm chứng:

1. **So sánh trên 1 lần chia train/test**: huấn luyện lại cả 3 mô hình với cấu hình đã chốt trên cùng một tập train/test, tính đầy đủ Accuracy, Precision, Recall, F1, AUC, Confusion Matrix, vẽ đường cong ROC.
2. **5-fold Stratified Cross-Validation**: chạy lại đúng cấu hình tốt nhất của mỗi mô hình qua 5 lần chia dữ liệu khác nhau, tính trung bình và độ lệch chuẩn của AUC/Recall – để xác nhận kết quả ở bước (1) không phải do may rủi của một cách chia dữ liệu cụ thể.

Chương 4

Kết quả thực nghiệm và xây dựng hệ thống demo

4.1 Kết quả dò tham số từng mô hình

4.1.1 Decision Tree

Phiên bản Decision Tree mặc định (không giới hạn độ sâu) cho thấy dấu hiệu overfitting rõ rệt: Accuracy trên tập huấn luyện đạt 100% trong khi trên tập kiểm tra chỉ 91,68%, với cây mọc sâu tới 18 tầng và 289 lá. Sau khi dò `max_depth` từ 1 đến 20, cấu hình tốt nhất được chọn là `max_depth=4, class_weight='balanced'`, đạt AUC = 0,8305 (Hình **??**).

4.1.2 k-Nearest Neighbors

Thử nghiệm ban đầu với $K = 5$ cho thấy Recall gần như bằng 0, do với chỉ ≈5% dữ liệu thuộc nhóm dương tính, xác suất tìm đủ đa số hàng xóm thuộc nhóm này rất thấp. Sau khi dò K từ 1 đến 30, cấu hình $K = 16$, `weights='uniform'` đạt AUC cao nhất = 0,7831, nhưng Recall vẫn bằng 0 ở ngưỡng mặc định 0,5 (AUC đo khả năng xếp hạng, không đo quyết định ở một ngưỡng cụ thể). Dò thêm ngưỡng quyết định, ngưỡng **0,06** được chọn để đạt Recall = 84%, tương đương Decision Tree, phục vụ so sánh công bằng.

4.1.3 Support Vector Machine

Dò 12 cấu hình (`kernel` × `C`), cấu hình tốt nhất là `kernel='linear', C=0,01, class_weight='balance`
đạt AUC = **0,8404** – cao nhất trong cả 3 mô hình – và đạt Recall = 84% ngay ở ngưỡng mặc định, không cần kỹ thuật hạ ngưỡng như k-NN.

Bảng 4.1: Cấu hình tốt nhất của từng mô hình

Mô hình	Tham số tốt nhất	AUC
Decision Tree	max_depth=4, class_weight='balanced'	0,8305
k-NN	K=16, weights='uniform', threshold=0,06	0,7831
SVM	**kernel='linear', C=0,01, class_weight='balanced'**	**0,8404**

4.2 So sánh tổng hợp và kết luận chính

Huấn luyện lại cả 3 mô hình với cấu hình đã chốt trên cùng tập train/test, canh cùng mức Recall = 84% (cùng bắt đúng 42/50 ca đột quỵ thật trong tập kiểm tra, bỏ sót 8/50 ca) để so sánh công bằng tuyệt đối (Bảng 4.2).

Bảng 4.2: So sánh tổng hợp 3 mô hình (cùng mức Recall = 0,84)

Mô hình	Accuracy	Precision	Recall	F1	AUC
SVM	**0,7260**	**0,1338**	0,84	**0,2308**	**0,8404**
Decision Tree	0,6526	0,1080	0,84	0,1913	0,8305
k-NN	0,6370	0,1037	0,84	0,1846	0,7831

Ở cùng mức Recall, **SVM thắng ở cả 4/4 độ đo còn lại**. Ba lý do giải thích kết quả này:

1. `class_weight='balanced'` của SVM tác động trực tiếp vào hàm mục tiêu khi huấn luyện, giống Decision Tree – trong khi k-NN phải dùng kỹ thuật "hạ ngưỡng sau khi huấn luyện", vốn kém tối ưu hơn.

2. Ranh giới phân tách 2 nhóm trong dữ liệu khá đơn giản (`kernel='linear'` thắng `'rbf'` ở mọi giá trị C), phù hợp với cách SVM tìm một siêu phẳng tối ưu toàn cục – trong khi k-NN chỉ dựa vào các điểm lân cận cục bộ, dễ bị nhiễu.

3. Tham số `C` nhỏ (mô hình "mềm dẻo") tổng quát hóa tốt hơn, nhất quán với bài học tránh overfitting rút ra từ Decision Tree.

Kết luận chính thức: Trong 3 mô hình đã thử nghiệm trên bộ dữ liệu Stroke Prediction, **SVM với kernel tuyến tính, C=0,01, class_weight='balanced' là mô hình tối ưu nhất**, cân bằng tốt nhất giữa khả năng phát hiện đúng ca có nguy cơ đột quỵ và độ chính xác tổng thể.

4.3 Kiểm chứng bằng 5-fold Cross-Validation

Để xác nhận kết quả trên không phải do may rủi của một cách chia dữ liệu, đồ án chạy bổ sung 5-fold Stratified Cross-Validation cho cấu hình tốt nhất của cả 3 mô hình (Bảng 4.3).

Bảng 4.3: Kết quả 5-fold Cross-Validation (trung bình ± độ lệch chuẩn qua 5 fold)

Mô hình	AUC (CV)	Recall (CV)
Decision Tree	0,7853 ± 0,0239	0,6985 ± 0,0929
k-NN	0,7098 ± 0,0200	0,7551 ± 0,0251
SVM	**0,8379 ± 0,0249**	0,8069 ± 0,0564

Kết quả cho thấy **thứ hạng 3 mô hình không đổi** (SVM > Decision Tree > k-NN) giữa 2 cách đánh giá, với độ lệch chuẩn AUC nhỏ (0,02–0,025) – xác nhận kết quả ổn định, không phải ngẫu nhiên, củng cố thêm độ tin cậy cho kết luận chính của đồ án.

4.4 Xây dựng hệ thống demo

4.4.1 Kiến trúc hệ thống

Hệ thống demo được xây dựng theo kiến trúc client–server, tách biệt phần xử lý mô hình (Backend) và phần giao diện (Frontend):

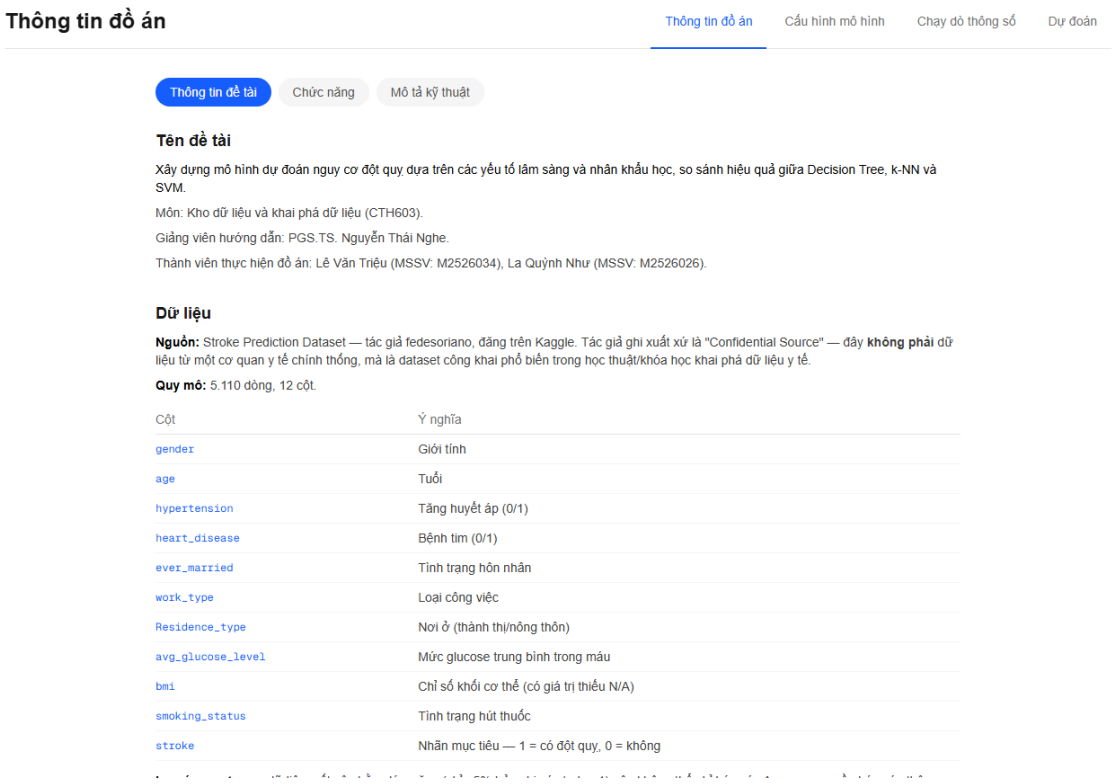
- **Backend – Python (scikit-learn, FastAPI)**: tiền xử lý dữ liệu, huấn luyện 3 mô hình, đóng gói thành các file `.pkl` (dùng thư viện `joblib`), và cung cấp API REST nhận thông tin bệnh nhân, trả về kết quả dự đoán của cả 3 mô hình.
- **Frontend – Next.js (React, TypeScript)**: giao diện web, gọi API Backend, hiển thị kết quả và biểu đồ minh chứng.

Luồng dữ liệu tổng thể: *Next.js (form nhập liệu) → gọi API FastAPI (chạy model đã train sẵn) → trả JSON kết quả 3 mô hình → Next.js hiển thị kết quả*.

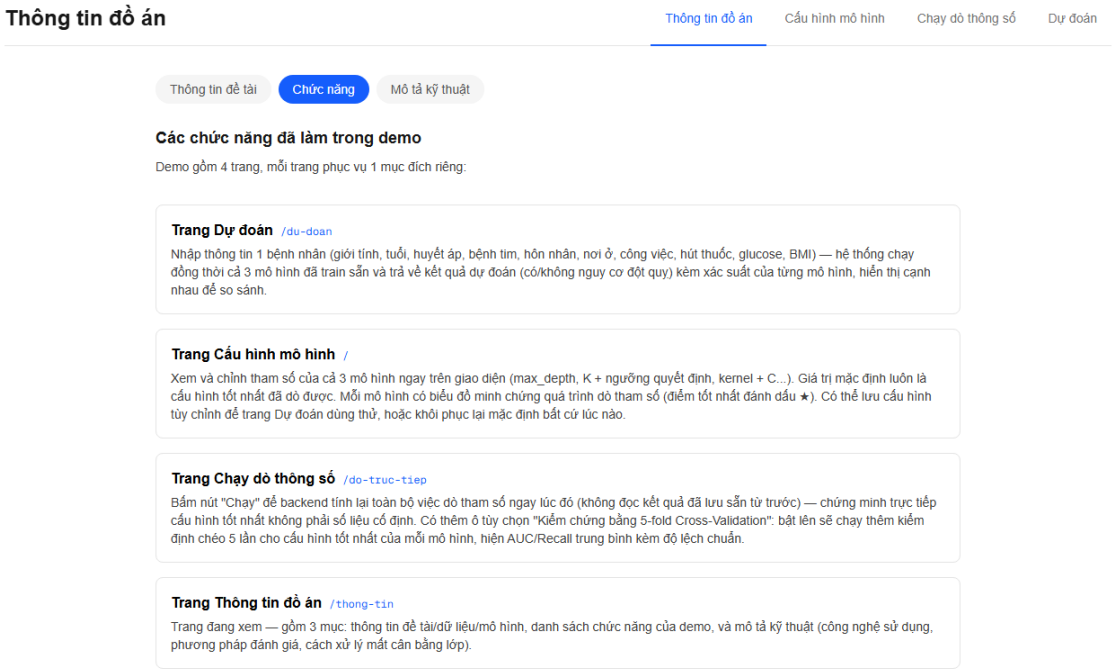
4.4.2 Các chức năng của hệ thống demo

Hệ thống gồm 4 trang chức năng, trình bày lần lượt dưới đây.

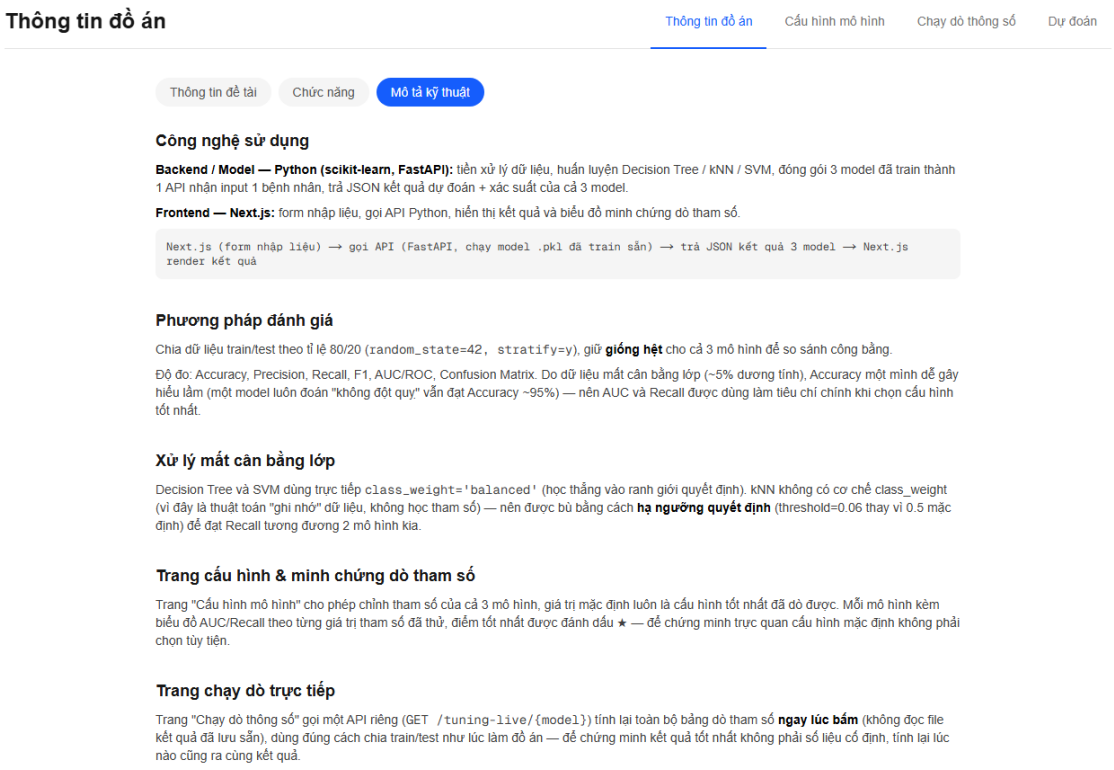
Trang Thông tin đồ án. Trang giới thiệu tổng quan, gồm 3 mục: thông tin đề tài/dữ liệu/mô hình (Hình 4.1), danh sách chức năng của hệ thống (Hình 4.2), và mô tả kỹ thuật (Hình 4.3).



Hình 4.1: Trang Thông tin đồ án – mục Thông tin đề tài

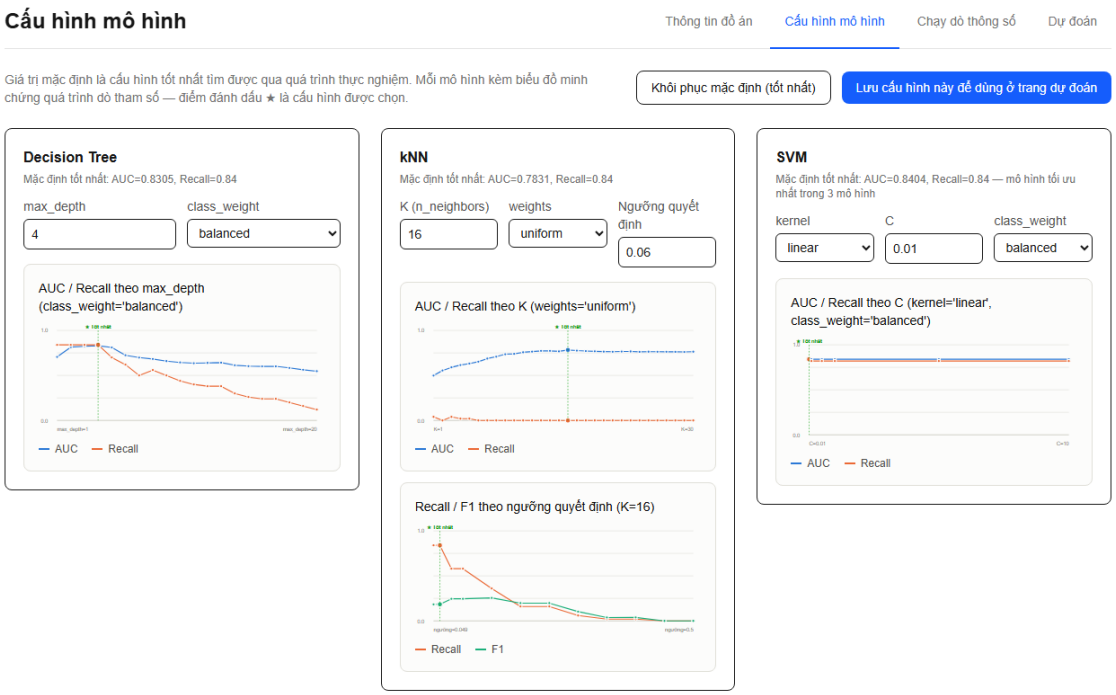


Hình 4.2: Trang Thông tin đồ án – mục Chức năng



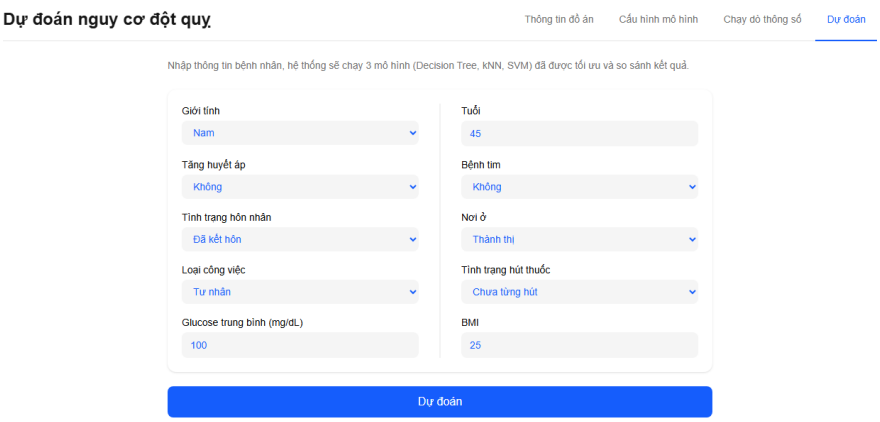
Hình 4.3: Trang Thông tin đồ án – mục Mô tả kỹ thuật

Trang Cấu hình mô hình. Cho phép người dùng xem và điều chỉnh tham số của cả 3 mô hình trực tiếp trên giao diện. Giá trị mặc định luôn là cấu hình tốt nhất đã dò được. Mỗi mô hình kèm biểu đồ AUC/Recall theo từng giá trị tham số đã thử, điểm tốt nhất được đánh dấu sao () – để chứng minh trực quan rằng cấu hình mặc định không phải chọn tùy tiện mà có thực nghiệm đứng sau (Hình 4.4).

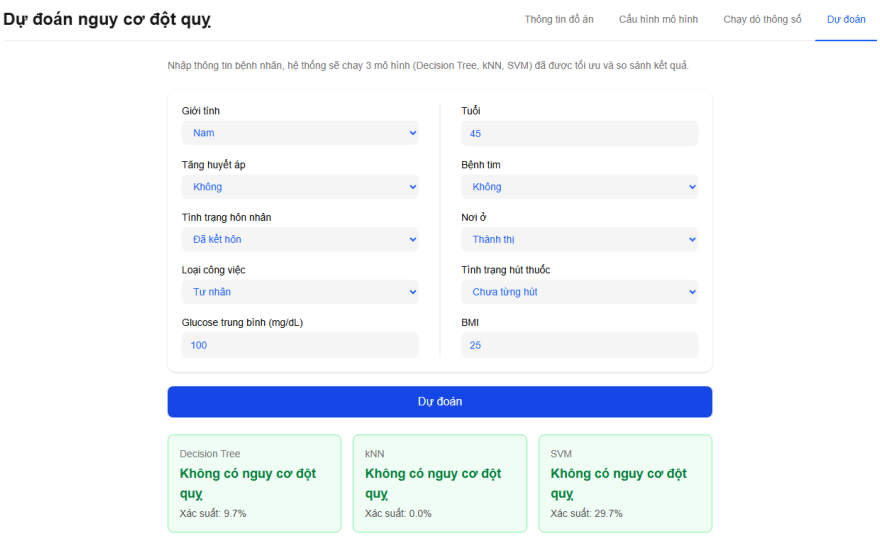


Hình 4.4: Trang Cấu hình mô hình, kèm biểu đồ minh chứng dò tham số

Trang Dự đoán. Người dùng nhập thông tin một bệnh nhân (giới tính, tuổi, huyết áp, bệnh tim, hôn nhân, nơi ở, công việc, hút thuốc, glucose, BMI) (Hình 4.5); hệ thống chạy đồng thời cả 3 mô hình và trả về kết quả dự đoán kèm xác suất của từng mô hình để so sánh (Hình 4.6).

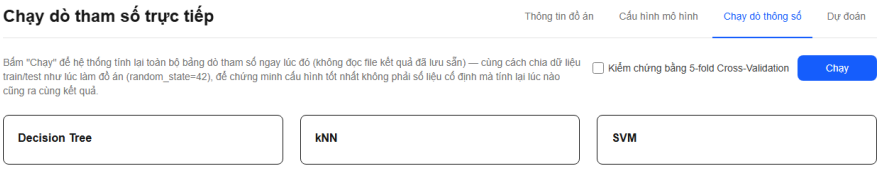


Hình 4.5: Trang Dự đoán – biểu mẫu nhập thông tin bệnh nhân

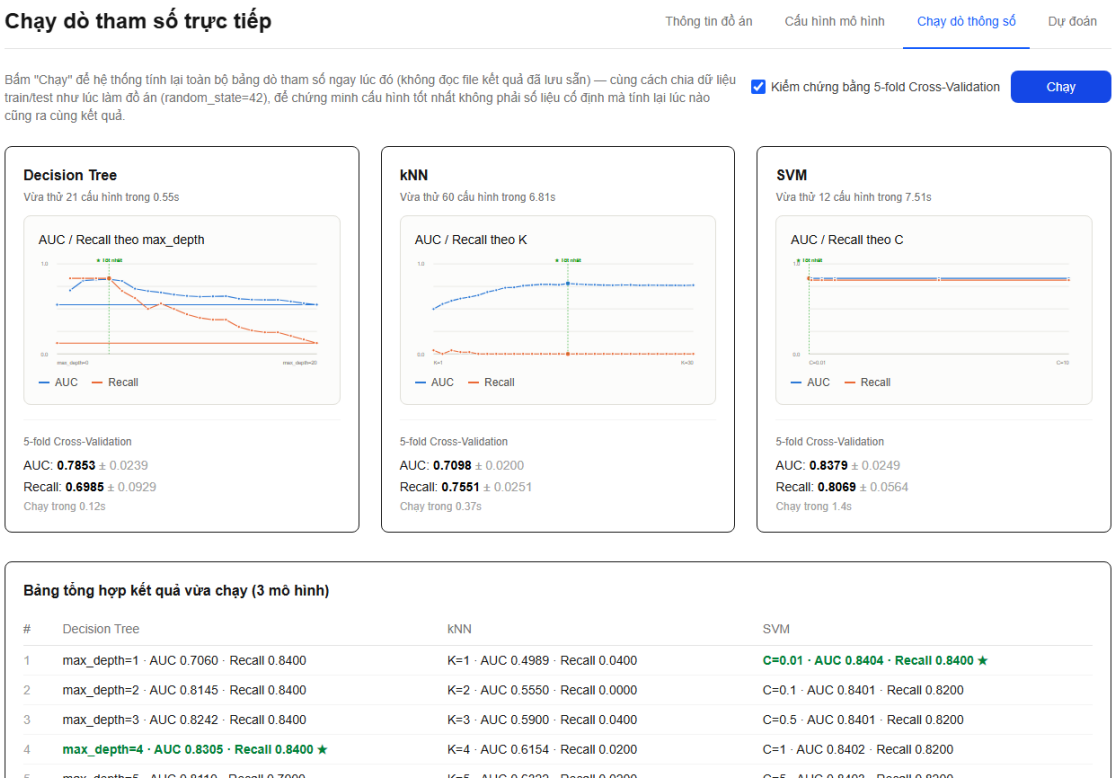


Hình 4.6: Trang Dự đoán – kết quả dự đoán từ cả 3 mô hình

Trang Chạy dò thông số. Điểm đặc biệt của hệ thống: khi bấm nút ”Chạy”, Backend **tính lại toàn bộ việc dò tham số ngay lúc đó** (không đọc kết quả đã lưu sẵn từ trước), cùng cách chia dữ liệu như lúc làm đồ án – chứng minh trực tiếp rằng cấu hình tốt nhất không phải số liệu cố định (Hình 4.7). Trang này còn có tùy chọn ”Kiểm chứng bằng 5-fold Cross-Validation”: khi bật, hệ thống chạy thêm kiểm định chéo 5 lần cho cấu hình tốt nhất của mỗi mô hình và hiển thị kết quả (Hình 4.8).



Hình 4.7: Trang Chạy dò thông số – trước khi bấm chạy



Hình 4.8: Trang Chạy dò thông số – kết quả sau khi chạy, kèm kiểm chứng 5-fold Cross-Validation

Chương 5

Kết luận và hướng phát triển

5.1 Kết luận

Đồ án đã hoàn thành đầy đủ các mục tiêu đề ra: xây dựng và dò tham số tối ưu cho 3 mô hình Decision Tree, k-NN và SVM trên bộ dữ liệu dự đoán đột quỵ; xử lý đúng đắn vấn đề mất cân bằng lớp bằng `class_weight='balanced'` và điều chỉnh ngưỡng quyết định; đánh giá khách quan bằng nhiều độ đo phù hợp (Precision, Recall, F1, AUC, Confusion Matrix) thay vì chỉ Accuracy; và kiểm chứng kết quả bằng 5-fold Cross-Validation.

Kết quả thực nghiệm khẳng định **SVM với kernel tuyến tính, C=0,01, class_weight='balanced'** là mô hình phù hợp nhất cho bài toán này, đạt AUC = 0,8404 (một lần chia train/test) và AUC = 0,8379 $\pm$ 0,0249 (5-fold Cross-Validation), vượt trội và ổn định hơn Decision Tree và k-NN. Đồ án cũng xây dựng thành công một hệ thống demo đầy đủ chức năng, cho phép người dùng (kể cả không chuyên về kỹ thuật) tương tác trực tiếp với cả 3 mô hình, tự điều chỉnh tham số, và quan trọng nhất là **tự kiểm chứng lại** quá trình dò tham số và kiểm định chéo bất cứ lúc nào, thay vì chỉ xem số liệu tĩnh.

5.2 Hạn chế

- Bộ dữ liệu sử dụng là dữ liệu công khai từ Kaggle, nguồn gốc không được công bố rõ ràng (ghi "Confidential Source"), nên kết quả mang tính minh họa phương pháp luận, chưa thể khẳng định độ chính xác lâm sàng trên dữ liệu bệnh viện thực tế.
- Do dữ liệu mất cân bằng lớp nặng, cả 3 mô hình đều có Precision thấp (10–13%) ở mức Recall 84% – nghĩa là có nhiều "báo động giả", cần cân nhắc thêm khi ứng dụng thực tế.
- Đồ án giới hạn trong 3 mô hình đã học ở môn học, chưa thử nghiệm các kỹ thuật cân bằng dữ liệu khác (như SMOTE) hay các mô hình phức tạp hơn (ensemble, deep learning).

5.3 Hướng phát triển

- Thử nghiệm thêm các kỹ thuật xử lý mất cân bằng lớp khác (SMOTE, undersampling) và so sánh với cách tiếp cận `class_weight` hiện tại.
- Mở rộng so sánh với các mô hình học máy khác (Random Forest, Gradient Boosting, mạng nơ-ron) để có cái nhìn toàn diện hơn.
- Nếu có điều kiện tiếp cận dữ liệu bệnh viện thực tế (có kiểm soát quyền riêng tư đúng quy định), kiểm chứng lại mô hình trên dữ liệu đó để đánh giá khả năng ứng dụng thực tiễn.
- Hoàn thiện hệ thống demo thành công cụ sàng lọc hỗ trợ bác sĩ, bổ sung xác thực người dùng và lưu trữ lịch sử dự đoán.

Tài liệu tham khảo

[1] fedesoriano, *Stroke Prediction Dataset*, Kaggle, 2021. `https://www.kaggle.com/datasets/fedesoriano/stroke-prediction-dataset`

[2] Pedregosa, F. et al., *Scikit-learn: Machine Learning in Python*, Journal of Machine Learning Research, 12, 2825–2830, 2011.

[3] World Health Organization, *The top 10 causes of death*, WHO Fact Sheets, 2020.